Solution: Converting Decimal to Binary

Problem Statement

Write a Recursive Procedure to Convert a Decimal Number to a Binary Equivalent.

Given a decimal number, we need to convert it to its binary representation, as explained in the following table:

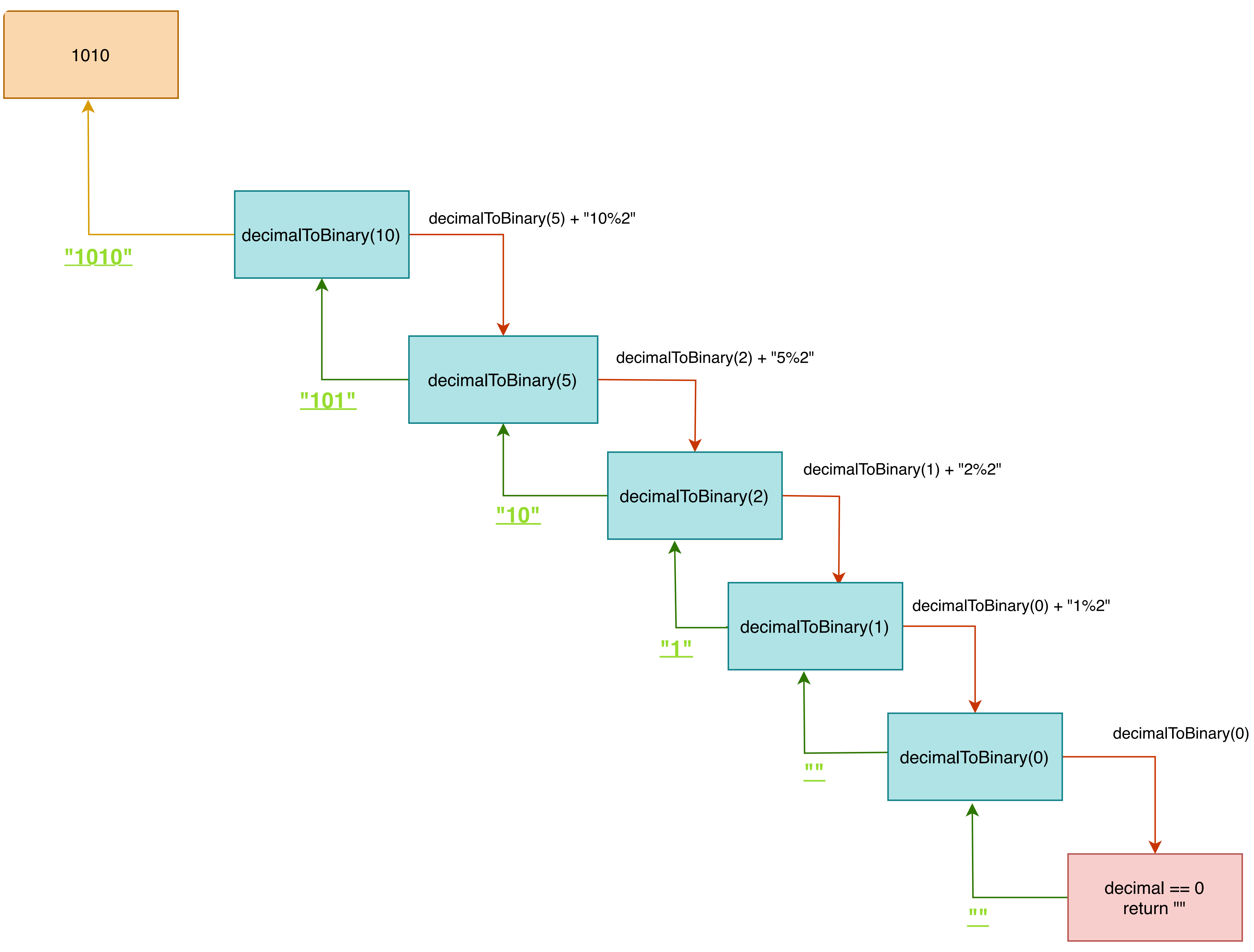
Input(s)	Output(s)	Explanation
Decimal Number = 10	Binary Number = 1010	The decimal number 10 can be represented as 1010 in binary. Each digit in the binary representation corresponds to the powers of 2.
Decimal Number = 27	Binary Number = 11011	The decimal number 27 can be represented as 11011 in binary. Each digit in the binary representation corresponds to the powers of 2.
Decimal Number = 5	Binary Number = 101	The decimal number 5 can be represented as 101 in binary. Each digit in the binary representation corresponds to the powers of 2.

Solution

The algorithm to convert a decimal number to a binary number using recursion can be defined as follows:

1. If the decimal number is 0, return an empty string (base case).
2. Otherwise, return the concatenation of the recursive call to convert the quotient of the decimal number divided by 2 to binary and the remainder of the decimal number divided by 2.

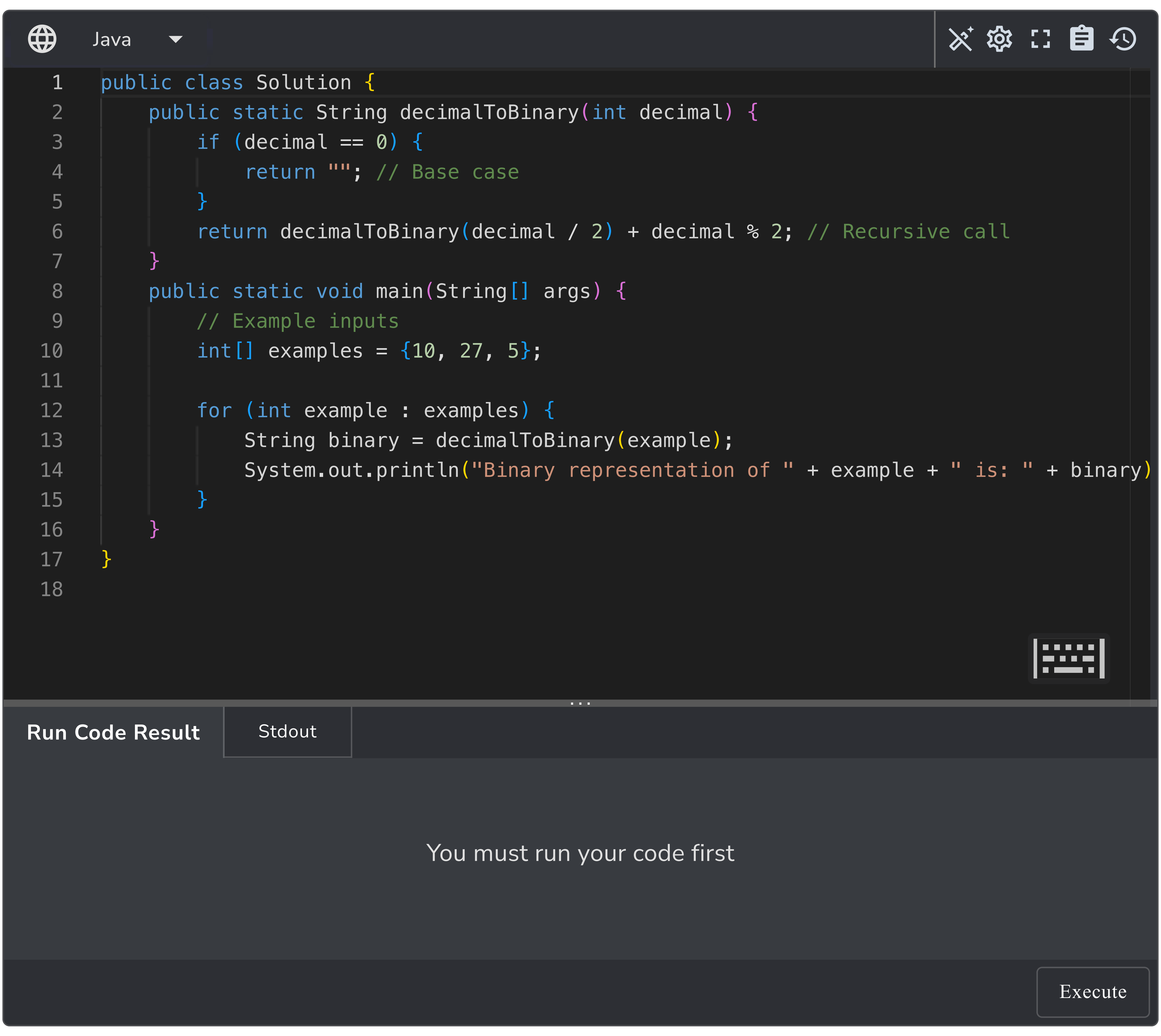
The recursive approach works by continuously dividing the decimal number by 2 and building the binary representation from the remainder. The base case is reached when the decimal number becomes 0. At this point, we have obtained the binary representation.


1010
"1010"
decimalToBinary(10)
decimalToBinary(5) + "10%2"
"101"
decimalToBinary(5)
decimalToBinary(2) + "5%2"
"10"
decimalToBinary(2)
decimalToBinary(1) + "2%2"
"1"
decimalToBinary(1)
decimalToBinary(0) + "1%2"
""
decimalToBinary(0)
decimalToBinary(0)
""
decimal == 0
return ""

Example dry run to calculate Binary of 10

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
public class Solution {
    public static String decimalToBinary(int decimal) {
        if (decimal == 0) {
            return ""; // Base case
        }
        return decimalToBinary(decimal / 2) + decimal % 2; // Recursive call
    }
    public static void main(String[] args) {
        // Example inputs
        int[] examples = {10, 27, 5};

        for (int example : examples) {
            String binary = decimalToBinary(example);
            System.out.println("Binary representation of " + example + " is: " + binary)
        }
    }
}
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(logN)$, where N is the decimal number. This is because the algorithm divides the decimal number by 2 in each recursive call until the base case is reached.

The space complexity is $O(logN)$ as well, as the maximum depth of the recursion is determined by the number of bits required to represent the decimal number in binary.